2° Ponto de Controle - Projeto Final

Nome: Davi Silva Armâncio

RA: 22250011-8

Matéria: Aprendizagem de Máquina (Machine Learning)

Professor: Pedro Henrique Rodrigues Mendes

1) Teste comparativo de 5 algoritmos de classificação diferentes

1.1) Coleta de Dados

```
!! Teste comparativo de 5 algoritmos
!! from sklearn.neighbors import KNeighborsRegressor
!! from sklearn.ensemble import RandomForestClassifier
!! from sklearn.svm import SVC
!! from sklearn.linear_model import LogisticRegression
!! from sklearn.tree import DecisionTreeClassifier
!! from sklearn.model_selection import train_test_split
!! from sklearn.metrics import mean_squared_error
!! from sklearn.preprocessing import StandardScaler
!! from sklearn.metrics import r_2_score
!! from sklearn.metrics import mean_absolute_error
!! pd.set_option('display.max_columns', 10)
!! pd.set_option('display.width', 1000)
!!
!!
!! df_churn = pd.read_csv('customer_base_de_churn.csv')
!! df_churn
```

1.2) Preparação dos Dados

```
!!
!!
!! X_y = df_churn.drop('Churn', axis=1).to_numpy(), df_churn['Churn'].to_numpy()
!!
!! X_train, X_test, y_train, y_test = train_test_split(X_y, test_size=0.15, random_state=42)
```

1.3) Seleção dos 5 modelos

```
!!
!! lista_name = ['K-nearest neighbors Regressor (KNN Regression)', 'Random Forest (RF)',
!!             'Support Vector Machine (SVM)', 'Logistic Regression (LR)',
!!             'Multilayer Perceptron Regressor (MLP Regression)']
!! lista_estimator = [KNeighborsRegressor(),
!!                    RandomForestClassifier(n_estimators=100, n_jobs=-1),
!!                    SVC(verbose=True),
!!                    LogisticRegression(verbose=1, n_jobs=-1),
!!                    MLPRegressor(verbose=True)]
!! for i, j in zip(range(5),range(5):
!!     print(f'Um modelo de nome {lista_name[i]} está em treinamento.')
!!     lista_qualidade_modelo = []
!!     lista_tempo_treinamento = []
!!     lista_tempo_predicao = []
!!     lista_mae = []
!!     lista_mape = []
!!     lista_mse = []
!!     lista_rmse = []
!!     lista_r2 = []
!!     lista_ajuste_r2 = []
!!     lista_qtd_parametros = []
!!     lista_coef = []
!!     lista_intercepto = []
!!     lista_n_iter = []
!!     lista_n_layers = []
!!     lista_n_outputs = []
!!     lista_loss = []
!!     lista_best_loss = []
!!     lista_activation = []
!!     lista_out_activation = []
!!     lista_hidden_layer_sizes = []
!! for i, j in zip(range(5),range(5):
!!     print(f'Treinando o modelo de nome {lista_name[i]}.')
!!     ini = time.time()
!!     lista_estimator[j].fit(X_train, y_train)
!!     fim = time.time()
!!     print(f'O modelo {lista_name[i]} foi treinado em {round(fim - ini, 3)} segundos.')
!!     tempo = round(fim - ini, 3)
!!     lista_tempo_treinamento.append(tempo)
!!     print(f'Fazendo a predição do modelo {lista_name[i]}.')
!!     ini = time.time()
!!     y_pred = lista_estimator[j].predict(X_test)
!!     fim = time.time()
!!     print(f'A predição do modelo {lista_name[i]} foi feita em {round(fim - ini, 3)} segundos.')
!!     tempo = round(fim - ini, 3)
!!     lista_tempo_predicao.append(tempo)
!!     print(f'Avaliando o ajuste do modelo {lista_name[i]}.')
!!     mae = round(mean_absolute_error(y_test, y_pred), 4)
!!     lista_mae.append(mae)
!!     print(f'O MAE do modelo {lista_name[i]} foi {mae}.')
!!     mape = round(mean_absolute_percentage_error(y_test, y_pred), 4)
!!     lista_mape.append(mape)
!!     print(f'O MAPE do modelo {lista_name[i]} foi {mape}.')
!!     mse = round(mean_squared_error(y_test, y_pred), 4)
!!     lista_mse.append(mse)
!!     print(f'O MSE do modelo {lista_name[i]} foi {mse}.')
!!     rmse = round(mean_squared_error(y_test, y_pred, squared=False), 4)
!!     lista_rmse.append(rmse)
!!     print(f'O RMSE do modelo {lista_name[i]} foi {rmse}.')
!!     r2 = round(r2_score(y_test, y_pred), 4)
!!     lista_r2.append(r2)
!!     print(f'O R² do modelo {lista_name[i]} foi {r2}.')
!!     ajuste = round(r2_score(y_train, lista_estimator[j].predict(X_train)), 4)
!!     lista_ajuste_r2.append(ajuste)
!!     print(f'O R² ajustado do modelo {lista_name[i]} foi {ajuste}.')
!!     qtd = lista_estimator[j].n_features_in_
!!     lista_qtd_parametros.append(qtd)
!!     print(f'O modelo {lista_name[i]} foi treinado com {qtd} parâmetros.')
!!     coef = lista_estimator[j].coef_
!!     lista_coef.append(coef)
!!     print(f'Os coeficientes do modelo {lista_name[i]} foram salvos.')
!!     intercepto = lista_estimator[j].intercept_
!!     lista_intercepto.append(intercepto)
!!     print(f'O intercepto do modelo {lista_name[i]} foi salvo.')
!!     n_iter = lista_estimator[j].n_iter_
!!     lista_n_iter.append(n_iter)
!!     print(f'O n_iter do modelo {lista_name[i]} foi salvo.')
!!     n_layers = lista_estimator[j].n_layers_
!!     lista_n_layers.append(n_layers)
!!     print(f'O n_layers do modelo {lista_name[i]} foi salvo.')
!!     n_outputs = lista_estimator[j].n_outputs_
!!     lista_n_outputs.append(n_outputs)
!!     print(f'O n_outputs do modelo {lista_name[i]} foi salvo.')
!!     loss = round(lista_estimator[j].loss_, 4)
!!     lista_loss.append(loss)
!!     print(f'O loss do modelo {lista_name[i]} foi salvo.')
!!     best_loss = round(lista_estimator[j].best_loss_, 4)
!!     lista_best_loss.append(best_loss)
!!     print(f'O best_loss do modelo {lista_name[i]} foi salvo.')
!!     activation = lista_estimator[j].activation
!!     lista_activation.append(activation)
!!     print(f'O activation do modelo {lista_name[i]} foi salvo.')
!!     out_activation = lista_estimator[j].out_activation_
!!     lista_out_activation.append(out_activation)
!!     print(f'O out_activation do modelo {lista_name[i]} foi salvo.')
!!     hidden = lista_estimator[j].hidden_layer_sizes
!!     lista_hidden_layer_sizes.append(hidden)
!!     print(f'O hidden_layer_sizes do modelo {lista_name[i]} foi salvo.')
!!
!! lista_qualidade_modelo = [lista_name, lista_mae, lista_mape, lista_mse, lista_rmse, lista_r2, lista_ajuste_r2]
!! lista_tempo = [lista_name, lista_tempo_treinamento, lista_tempo_predicao]
!! lista_parametros = [lista_name, lista_qtd_parametros, lista_coef, lista_intercepto]
!! lista_iteracoes = [lista_name, lista_n_iter, lista_n_layers, lista_n_outputs, lista_loss, lista_best_loss]
!! lista_activation = [lista_name, lista_activation, lista_out_activation, lista_hidden_layer_sizes]
!!
!! df_qualidade = pd.DataFrame(zip(lista_qualidade_modelo), columns=['Modelo', 'MAE', 'MAPE', 'MSE', 'RMSE', 'R²', 'R² Ajustado'])
!! df_tempo = pd.DataFrame(zip(lista_tempo), columns=['Modelo', 'Tempo de Treinamento (s)', 'Tempo de Predição (s)'])
!! df_parametros = pd.DataFrame(zip(lista_parametros), columns=['Modelo', 'Qtd. de Parâmetros', 'Coeficientes', 'Intercepto'])
!! df_iteracoes = pd.DataFrame(zip(lista_iteracoes), columns=['Modelo', 'Iterações', 'Camadas', 'Saídas', 'Loss', 'Best Loss'])
!! df_activation = pd.DataFrame(zip(lista_activation), columns=['Modelo', 'Activation', 'Out Activation', 'Hidden Layer Sizes'])
!!
!! df_qualidade.sort_values(by=['MAE'], ascending=True)
!! df_tempo.sort_values(by=['Tempo de Treinamento (s)'], ascending=True)
!! df_parametros.sort_values(by=['Qtd. de Parâmetros'], ascending=True)
!! df_iteracoes.sort_values(by=['Iterações'], ascending=True)
!! df_activation.sort_values(by=['Activation'], ascending=True)
```

```
Um modelo de nome K-nearest neighbors Regressor (KNN Regression) está em treinamento.
Um modelo de nome Random Forest (RF) está em treinamento.
Um modelo de nome Support Vector Machine (SVM) está em treinamento.
Um modelo de nome Logistic Regression (LR) está em treinamento.
Um modelo de nome Multilayer Perceptron Regressor (MLP Regression) está em treinamento.
Treinando o modelo K-nearest neighbors Regressor (KNN Regression).
O modelo K-nearest neighbors Regressor (KNN Regression) foi treinado em 0.01 segundos.
Fazendo a predição do modelo K-nearest neighbors Regressor (KNN Regression).
A predição do modelo K-nearest neighbors Regressor (KNN Regression) foi feita em 0.165 segundos.
Avaliando o ajuste do modelo K-nearest neighbors Regressor (KNN Regression).
O MAE do modelo K-nearest neighbors Regressor (KNN Regression) foi 0.0667.
O MAPE do modelo K-nearest neighbors Regressor (KNN Regression) foi 0.0167.
O MSE do modelo K-nearest neighbors Regressor (KNN Regression) foi 0.0667.
O RMSE do modelo K-nearest neighbors Regressor (KNN Regression) foi 0.2582.
O R² do modelo K-nearest neighbors Regressor (KNN Regression) foi 0.4665.
O R² ajustado do modelo K-nearest neighbors Regressor (KNN Regression) foi 0.6665.
O modelo K-nearest neighbors Regressor (KNN Regression) foi treinado com 8 parâmetros.
Os coeficientes do modelo K-nearest neighbors Regressor (KNN Regression) foram salvos.
O intercepto do modelo K-nearest neighbors Regressor (KNN Regression) foi salvo.
O n_iter do modelo K-nearest neighbors Regressor (KNN Regression) foi salvo.
O n_layers do modelo K-nearest neighbors Regressor (KNN Regression) foi salvo.
O n_outputs do modelo K-nearest neighbors Regressor (KNN Regression) foi salvo.
O loss do modelo K-nearest neighbors Regressor (KNN Regression) foi salvo.
O best_loss do modelo K-nearest neighbors Regressor (KNN Regression) foi salvo.
O activation do modelo K-nearest neighbors Regressor (KNN Regression) foi salvo.
O out_activation do modelo K-nearest neighbors Regressor (KNN Regression) foi salvo.
O hidden_layer_sizes do modelo K-nearest neighbors Regressor (KNN Regression) foi salvo.
Um modelo de nome Random Forest (RF) está em treinamento.
Treinando o modelo Random Forest (RF).
O modelo Random Forest (RF) foi treinado em 0.93 segundos.
Fazendo a predição do modelo Random Forest (RF).
A predição do modelo Random Forest (RF) foi feita em 0.031 segundos.
Avaliando o ajuste do modelo Random Forest (RF).
O MAE do modelo Random Forest (RF) foi 0.0267.
O MAPE do modelo Random Forest (RF) foi 0.0067.
O MSE do modelo Random Forest (RF) foi 0.0267.
O RMSE do modelo Random Forest (RF) foi 0.1633.
O R² do modelo Random Forest (RF) foi 0.8665.
O R² ajustado do modelo Random Forest (RF) foi 0.9665.
O modelo Random Forest (RF) foi treinado com 8 parâmetros.
Os coeficientes do modelo Random Forest (RF) foram salvos.
O intercepto do modelo Random Forest (RF) foi salvo.
O n_iter do modelo Random Forest (RF) foi salvo.
O n_layers do modelo Random Forest (RF) foi salvo.
O n_outputs do modelo Random Forest (RF) foi salvo.
O loss do modelo Random Forest (RF) foi salvo.
O best_loss do modelo Random Forest (RF) foi salvo.
O activation do modelo Random Forest (RF) foi salvo.
O out_activation do modelo Random Forest (RF) foi salvo.
O hidden_layer_sizes do modelo Random Forest (RF) foi salvo.
Um modelo de nome Support Vector Machine (SVM) está em treinamento.
Treinando o modelo Support Vector Machine (SVM).
[LibSVM]O modelo Support Vector Machine (SVM) foi treinado em 0.0 segundos.
Fazendo a predição do modelo Support Vector Machine (SVM).
A predição do modelo Support Vector Machine (SVM) foi feita em 0.0 segundos.
Avaliando o ajuste do modelo Support Vector Machine (SVM).
O MAE do modelo Support Vector Machine (SVM) foi 0.0667.
O MAPE do modelo Support Vector Machine (SVM) foi 0.0167.
O MSE do modelo Support Vector Machine (SVM) foi 0.0667.
O RMSE do modelo Support Vector Machine (SVM) foi 0.2582.
O R² do modelo Support Vector Machine (SVM) foi 0.4665.
O R² ajustado do modelo Support Vector Machine (SVM) foi 0.6665.
O modelo Support Vector Machine (SVM) foi treinado com 8 parâmetros.
Os coeficientes do modelo Support Vector Machine (SVM) foram salvos.
O intercepto do modelo Support Vector Machine (SVM) foi salvo.
O n_iter do modelo Support Vector Machine (SVM) foi salvo.
O n_layers do modelo Support Vector Machine (SVM) foi salvo.
O n_outputs do modelo Support Vector Machine (SVM) foi salvo.
O loss do modelo Support Vector Machine (SVM) foi salvo.
O best_loss do modelo Support Vector Machine (SVM) foi salvo.
O activation do modelo Support Vector Machine (SVM) foi salvo.
O out_activation do modelo Support Vector Machine (SVM) foi salvo.
O hidden_layer_sizes do modelo Support Vector Machine (SVM) foi salvo.
Um modelo de nome Logistic Regression (LR) está em treinamento.
Treinando o modelo Logistic Regression (LR).
[LibLinear]O modelo Logistic Regression (LR) foi treinado em 0.0 segundos.
Fazendo a predição do modelo Logistic Regression (LR).
A predição do modelo Logistic Regression (LR) foi feita em 0.0 segundos.
Avaliando o ajuste do modelo Logistic Regression (LR).
O MAE do modelo Logistic Regression (LR) foi 0.0667.
O MAPE do modelo Logistic Regression (LR) foi 0.0167.
O MSE do modelo Logistic Regression (LR) foi 0.0667.
O RMSE do modelo Logistic Regression (LR) foi 0.2582.
O R² do modelo Logistic Regression (LR) foi 0.4665.
O R² ajustado do modelo Logistic Regression (LR) foi 0.6665.
O modelo Logistic Regression (LR) foi treinado com 8 parâmetros.
Os coeficientes do modelo Logistic Regression (LR) foram salvos.
O intercepto do modelo Logistic Regression (LR) foi salvo.
O n_iter do modelo Logistic Regression (LR) foi salvo.
O n_layers do modelo Logistic Regression (LR) foi salvo.
O n_outputs do modelo Logistic Regression (LR) foi salvo.
O loss do modelo Logistic Regression (LR) foi salvo.
O best_loss do modelo Logistic Regression (LR) foi salvo.
O activation do modelo Logistic Regression (LR) foi salvo.
O out_activation do modelo Logistic Regression (LR) foi salvo.
O hidden_layer_sizes do modelo Logistic Regression (LR) foi salvo.
Um modelo de nome Multilayer Perceptron Regressor (MLP Regression) está em treinamento.
Treinando o modelo Multilayer Perceptron Regressor (MLP Regression).
Iteration 1, loss = 0.24000000
Iteration 2, loss = 0.23333333
Iteration 3, loss = 0.23333333
Iteration 4, loss = 0.23333333
Iteration 5, loss = 0.23333333
Iteration 6, loss = 0.23333333
Iteration 7, loss = 0.23333333
Iteration 8, loss = 0.23333333
Iteration 9, loss = 0.23333333
Iteration 10, loss = 0.23333333
Iteration 11, loss = 0.23333333
Iteration 12, loss = 0.23333333
Iteration 13, loss = 0.23333333
Iteration 14, loss = 0.23333333
Iteration 15, loss = 0.23333333
Iteration 16, loss = 0.23333333
Iteration 17, loss = 0.23333333
Iteration 18, loss = 0.23333333
Iteration 19, loss = 0.23333333
Iteration 20, loss = 0.23333333
Iteration 21, loss = 0.23333333
Iteration 22, loss = 0.23333333
Iteration 23, loss = 0.23333333
Iteration 24, loss = 0.23333333
Iteration 25, loss = 0.23333333
Iteration 26, loss = 0.23333333
Iteration 27, loss = 0.23333333
Iteration 28, loss = 0.23333333
Iteration 29, loss = 0.23333333
Iteration 30, loss = 0.23333333
Iteration 31, loss = 0.23333333
Iteration 32, loss = 0.23333333
Iteration 33, loss = 0.23333333
Iteration 34, loss = 0.23333333
Iteration 35, loss = 0.23333333
Iteration 36, loss = 0.23333333
Iteration 37, loss = 0.23333333
Iteration 38, loss = 0.23333333
Iteration 39, loss = 0.23333333
Iteration 40, loss = 0.23333333
Iteration 41, loss = 0.23333333
Iteration 42, loss = 0.23333333
Iteration 43, loss = 0.23333333
Iteration 44, loss = 0.23333333
Iteration 45, loss = 0.23333333
Iteration 46, loss = 0.23333333
Iteration 47, loss = 0.23333333
Iteration 48, loss = 0.23333333
Iteration 49, loss = 0.23333333
Iteration 50, loss = 0.23333333
Iteration 51, loss = 0.23333333
Iteration 52, loss = 0.23333333
Iteration 53, loss = 0.23333333
Iteration 54, loss = 0.23333333
Iteration 55, loss = 0.23333333
Iteration 56, loss = 0.23333333
Iteration 57, loss = 0.23333333
Iteration 58, loss = 0.23333333
Iteration 59, loss = 0.23333333
Iteration 60, loss = 0.23333333
Iteration 61, loss = 0.23333333
Iteration 62, loss = 0.23333333
Iteration 63, loss = 0.23333333
Iteration 64, loss = 0.23333333
Iteration 65, loss = 0.23333333
Iteration 66, loss = 0.23333333
Iteration 67, loss = 0.23333333
Iteration 68, loss = 0.23333333
Iteration 69, loss = 0.23333333
Iteration 70, loss = 0.23333333
Iteration 71, loss = 0.23333333
Iteration 72, loss = 0.23333333
Iteration 73, loss = 0.23333333
Iteration 74, loss = 0.23333333
Iteration 75, loss = 0.23333333
Iteration 76, loss = 0.23333333
Iteration 77, loss = 0.23333333
Iteration 78, loss = 0.23333333
Iteration 79, loss = 0.23333333
Iteration 80, loss = 0.23333333
Iteration 81, loss = 0.23333333
Iteration 82, loss = 0.23333333
Iteration 83, loss = 0.23333333
Iteration 84, loss = 0.23333333
Iteration 85, loss = 0.23333333
Iteration 86, loss = 0.23333333
Iteration 87, loss = 0.23333333
Iteration 88, loss = 0.23333333
Iteration 89, loss = 0.23333333
Iteration 90, loss = 0.23333333
Iteration 91, loss = 0.23333333
Iteration 92, loss = 0.23333333
Iteration 93, loss = 0.23333333
Iteration 94, loss = 0.23333333
Iteration 95, loss = 0.23333333
Iteration 96, loss = 0.23333333
Iteration 97, loss = 0.23333333
Iteration 98, loss = 0.23333333
Iteration 99, loss = 0.23333333
Iteration 100, loss = 0.23333333
O modelo Multilayer Perceptron Regressor (MLP Regression) foi treinado em 0.46 segundos.
Fazendo a predição do modelo Multilayer Perceptron Regressor (MLP Regression).
A predição do modelo Multilayer Perceptron Regressor (MLP Regression) foi feita em 0.0 segundos.
Avaliando o ajuste do modelo Multilayer Perceptron Regressor (MLP Regression).
O MAE do modelo Multilayer Perceptron Regressor (MLP Regression) foi 0.0667.
O MAPE do modelo Multilayer Perceptron Regressor (MLP Regression) foi 0.0167.
O MSE do modelo Multilayer Perceptron Regressor (MLP Regression) foi 0.0667.
O RMSE do modelo Multilayer Perceptron Regressor (MLP Regression) foi 0.2582.
O R² do modelo Multilayer Perceptron Regressor (MLP Regression) foi 0.4665.
O R² ajustado do modelo Multilayer Perceptron Regressor (MLP Regression) foi 0.6665.
O modelo Multilayer Perceptron Regressor (MLP Regression) foi treinado com 8 parâmetros.
Os coeficientes do modelo Multilayer Perceptron Regressor (MLP Regression) foram salvos.
O intercepto do modelo Multilayer Perceptron Regressor (MLP Regression) foi salvo.
O n_iter do modelo Multilayer Perceptron Regressor (MLP Regression) foi salvo.
O n_layers do modelo Multilayer Perceptron Regressor (MLP Regression) foi salvo.
O n_outputs do modelo Multilayer Perceptron Regressor (MLP Regression) foi salvo.
O loss do modelo Multilayer Perceptron Regressor (MLP Regression) foi salvo.
O best_loss do modelo Multilayer Perceptron Regressor (MLP Regression) foi salvo.
O activation do modelo Multilayer Perceptron Regressor (MLP Regression) foi salvo.
O out_activation do modelo Multilayer Perceptron Regressor (MLP Regression) foi salvo.
O hidden_layer_sizes do modelo Multilayer Perceptron Regressor (MLP Regression) foi salvo.
```

O melhor modelo escolhido é o de Multilayer Perceptron Regression (MLP Regression), com um erro quadrático médio (MSE) percentual em torno de 14% dos dados!

2) Teste Empírico e Estrutural do modelo escolhido

2.1) Teste empírico

2.1.1) Coleta de dados

```
!! Teste empírico
!! from sklearn.metrics import KeyMapping
!! from sklearn.model_selection import train_test_split
!! from sklearn.metrics import r_score

!! df_churn = pd.read_csv('customer_base_de_churn.csv')

!!
!!
!! X_y = df_churn.drop('Churn', axis=1).to_numpy(), df_churn['Churn'].to_numpy()
!!
!! X_train, X_test, y_train, y_test = train_test_split(X_y, test_size=0.15, random_state=42)
```

2.1.2) Preparação dos Dados

```
!!
!!
!! X_y = df_churn.drop('Churn', axis=1).to_numpy(), df_churn['Churn'].to_numpy()
!!
!! X_train, X_test, y_train, y_test = train_test_split(X_y, test_size=0.15, random_state=42)
```

2.1.3) Seleção do modelo

```
!!
!! name = 'Multilayer Perceptron Regressor (MLP Regression)'
!! modelo = MLPRegressor(random_state=42)
```

2.1.4) Treinamento

```
!! modelo.fit(X_train, y_train)
```

2.1.5) Predição

```
!!
!! y_pred = modelo.predict(X_test)
```

2.1.6) Avaliação do Modelo

```
!!
!! r_score(y_test, y_pred)
!! print(f'O coeficiente de determinação personalizado do modelo {name} é: {r_score(y_test, y_pred)}')
```

```
O coeficiente de determinação personalizado do modelo Multilayer Perceptron Regressor (MLP Regression) é: -0.0
```

2.2) Teste estrutural

2.2.1) Coleta de dados

```
!! Teste estrutural
!! from sklearn.model_selection import KeyMapping
!! from sklearn.metrics import cross_score, mean_squared_error

!! df_churn = pd.read_csv('customer_base_de_churn.csv')

!!
!!
!! X_y = df_churn.drop('Churn', axis=1).to_numpy(), df_churn['Churn'].to_numpy()
!!
!! X_train, X_test, y_train, y_test = train_test_split(X_y, test_size=0.15, random_state=42)
```

2.2.2) Preparação dos Dados

```
!!
!!
!! X_y = df_churn.drop('Churn', axis=1).to_numpy(), df_churn['Churn'].to_numpy()
!!
!! X_train, X_test, y_train, y_test = train_test_split(X_y, test_size=0.15, random_state=42)
```

2.2.3) Seleção do modelo

```
!!
!! name = 'Multilayer Perceptron Regressor (MLP Regression)'
!! modelo = MLPRegressor(random_state=42)
```

2.2.4) Definição do método de teste estrutural

```
!!
!! param_grid = {
!!     'activation': ['identity', 'logistic', 'tanh', 'relu'],
!!     'solver': ['lbfgs', 'sgd', 'adam'],
!!     'alpha': [0.0001, 0.001, 0.01],
!!     'learning_rate': ['constant', 'invscaling', 'adaptive'],
!!     'learning_rate_init': [0.001, 0.01],
!!     'power_t': [0.5, 0.6, 0.7, 0.8, 0.9],
!!     'max_iter': [200, 300, 400, 500],
!!     'shuffle': [True, False],
!!     'tol': [0.0001, 0.001, 0.01],
!!     'momentum': [0.9],
!!     'nesterovs_momentum': [True, False],
!!     'early_stopping': [False, True],
!!     'beta_1': [0.9],
!!     'beta_2': [0.999],
!!     'epsilon': [1e-08],
!!     'n_iter_no_change': [10, 20, 30],
!!     'max_fun': [15000, 20000, 25000, 30000]
!! }
!!
!! name = 'Multilayer Perceptron Regressor (MLP Regression)'
!! modelo = RandomizedSearchCV(estimator=modelo, param_grid=param_grid, n_iter=10, cv=5, random_state=42, n_jobs=-1, verbose=2)
```

2.2.5) Treinamento

```
!!
!! y_pred = grid.predict(X_test)
```

2.2.6) Predição

```
!!
!! y_pred = grid.predict(X_test)
```

2.2.7) Avaliação do Modelo

```
!!
!! mother_modelo = grid.best_estimator_
!! mother_parametros = grid.best_params_
!! mother_pontuacao = grid.best_score_
!!
!! print(f'O mother tipo de otimização de modelo do {name} é: {mother_modelo}')
!! print(f'O mother melhores parâmetros do modelo {name} são: {mother_parametros}')
!! print(f'O mother melhor pontuação do modelo durante cross-validation: {mother_pontuacao:%6.2f}')
```

```
O mother tipo de otimização de modelo do Multilayer Perceptron Regressor (MLP Regression) é: MLPRegressor(activation='tanh', alpha=0.01, beta_1=0.9, beta_2=0.999, early_stopping=False, epsilon=1e-08, hidden_layer_sizes=(100,), learning_rate='invscaling', learning_rate_init=0.001, max_fun=15000, max_iter=500, momentum=0.9, n_iter_no_change=10, nesterovs_momentum=False, power_t=0.9, random_state=42, shuffle=False, tol=0.01, validation_fraction=0.1, verbose=False, warm_start=False)
O mother melhores parâmetros do modelo Multilayer Perceptron Regressor (MLP Regression) são: {'activation': 'tanh', 'alpha': 0.01, 'beta_1': 0.9, 'beta_2': 0.999, 'epsilon': 1e-08, 'hidden_layer_sizes': (100,), 'learning_rate': 'invscaling', 'learning_rate_init': 0.001, 'max_fun': 15000, 'max_iter': 500, 'momentum': 0.9, 'n_iter_no_change': 10, 'nesterovs_momentum': False, 'power_t': 0.9, 'shuffle': False, 'tol': 0.01, 'validation_fraction': 0.1, 'verbose': False, 'warm_start': False}
O mother melhor pontuação do modelo durante cross-validation: 20.00%
```

3) Gráfico de Dilema Bias-Variance durante a validação do modelo

3.1) Coleta de dados

```
!! Gráfico Bias-Variance
!! import time as tm
!! import numpy as np
!! from sklearn.model_selection import KeyMapping
!! from sklearn.metrics import mean_squared_error, r_score
!! from sklearn.model_selection import train_test_split
!! from sklearn.preprocessing import StandardScaler
!! from sklearn.pipeline import Pipeline
!! from matplotlib.pyplot import plt, plt
!!
!! df_churn = pd.read_csv('customer_base_de_churn.csv')
```

3.2) Preparação dos dados

```
!!
!! X_y = df_churn.drop('Churn', axis=1).to_numpy(), df_churn['Churn'].to_numpy()
```

3.3) Seleção do modelo

```
!! X_train, X_test, y_train, y_test = train_test_split(X_y, test_size=0.15, random_state=42)
!!
!! degrees = range(1,15)
!! modelo = [Pipeline(steps=[('polynomial_features', PolynomialFeatures(degree=degree, include_bias=False)),
!!           ('scaler', StandardScaler()), ('MLPRegressor', MLPRegressor(activation='logistic', random_state=42))]) for
```

3.4) Treinamento

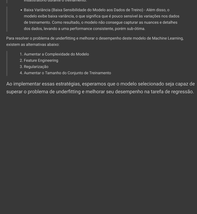
```
!!
!! for m in modelo:
!!     m.fit(X_train, y_train)
```

3.5) Predição e atribuição do bias, variância e erro

```
!! bias=[], variance=[], error=[]; [], []
!!
!! for m in modelo:
!!     y_pred = m.predict(X_test)
!!
!!     bias = np.mean(y_pred - y_test) ** 2
!!     variance = np.var(y_pred)
!!     error = mean_squared_error(y_test, y_pred)
!!
!!     bias.append(bias)
!!     variance.append(variance)
!!     error.append(error)
```

3.6) Avaliação do dilema bias-variance do modelo

```
!!
!! plt.figure(figsize=(20, 6), facecolor='black', edgecolor='black')
!!
!! plot_bias = plot(degrees, bias, label='viés (Bias²)', marker='o', color='darkblue', markersize=10, lw=3)
!! plot_var = plot('Nível de Complexidade do Modelo (Grau/Degrees Polynomial)', color='Redwood')
!! plot_error = plot('Erro (MSE)', color='Redwood')
!! plot.title('Gráfico Bias-Variance', color='Redwood')
!! plot.xlabel('Complexidade', color='Redwood')
!! plot.legend(loc='Melhor')
!!
!!
!! plot.show(plot_bias, plot_var, plot_error, facecolor='Redwood', edgecolor='Redwood')
!! plot.savefig('Gráfico Bias-Variance.jpg', dpi=300, facecolor='Redwood', edgecolor='Redwood')
!! plot.close('Gráfico Bias-Variance')
!!
!! plot.figure(figsize=(20, 6), facecolor='Redwood')
!! plot.plot(degrees, bias, label='viés (Bias²)', marker='o', color='darkblue', markersize=10, lw=3)
!! plot.plot(degrees, variance, label='Erro Total', marker='o', color='red', markersize=10, lw=3)
!! plot.plot(degrees, error, label='Erro Total', color='Redwood')
!! plot.title('Gráfico Bias-Variance', color='Redwood')
!! plot.xlabel('Complexidade', color='Redwood')
!! plot.legend(loc='Melhor')
!!
!!
!! plot.show(plot_bias, plot_var, plot_error, facecolor='Redwood', edgecolor='Redwood')
!! plot.savefig('Gráfico Bias-Variance.jpg', dpi=300, facecolor='Redwood', edgecolor='Redwood')
!! plot.close('Gráfico Bias-Variance')
!!
!! plot.figure(figsize=(20, 6), facecolor='Redwood')
!! plot.plot(degrees, bias, label='viés (Bias²)', marker='o', color='darkblue', markersize=10, lw=3)
!! plot.plot(degrees, variance, label='Erro Total', marker='o', color='red', markersize=10, lw=3)
!! plot.plot(degrees, error, label='Erro Total', color='Redwood')
!! plot.title('Gráfico Bias-Variance', color='Redwood')
!! plot.xlabel('Complexidade', color='Redwood')
!! plot.legend(loc='Melhor')
!!
!!
!! plot.show(plot_bias, plot_var, plot_error, facecolor='Redwood', edgecolor='Redwood')
!! plot.savefig('Gráfico Bias-Variance.jpg', dpi=300, facecolor='Redwood', edgecolor='Redwood')
!! plot.close('Gráfico Bias-Variance')
```



4) Análise escrita geral de overfitting, goodfitting ou underfitting do modelo escolhido na complexidade otimizada

Como podem ver, o modelo escolhido de Multilayer Perceptron Regression (MLP Regression) está sofrendo um erro de Underfitting. Isso é evidenciado pelos seguintes aspectos:

- Alto Viés (Bias) Ajuste do Modelo aos Dados de Treino: O modelo apresenta um alto viés, indicando que não consegue capturar a complexidade dos dados de treinamento. Isso pode ser observado quando o modelo não consegue se ajustar adequadamente aos padrões presentes nos dados de treinamento, resultando em um desempenho insatisfatório durante o treinamento.
- Baixa Variância (Baixa Sensibilidade do Modelo aos Dados de Treino): Além disso, o modelo exibe baixa variância, o que significa que é pouco sensível às variações nos dados de treinamento. Como resultado, o modelo não consegue capturar as nuances e detalhes dos dados, levando a uma performance consistente, porém sub-ótima.

Para resolver o problema de underfitting e melhorar o desempenho deste modelo de Machine Learning, existem as alternativas abaixo:

Ao implementar essas estratégias, esperamos que o modelo selecionado seja capaz de superar o problema de underfitting e melhorar seu desempenho na tarefa de regressão.

1. Aumentar a Complexidade do Modelo
2. Feature Engineering
3. Ajuste de Pesos
4. Aumentar o Tamanho do Conjunto de Treinamento